CAUSAL EVIDENCE OF STACK REPRESENTATIONS IN MODELING COUNTER LANGUAGES USING TRANSFORMERS

Anonymous authors
Paper under double-blind review

ABSTRACT

Formal languages have proven to be effective conduits to understand the inner mechanisms of transformers. Past work has shown that transformers trained on next-token prediction over counter languages learn representations consistent with an underlying stack structure. Beyond representational analysis, this paper investigates the causal role of these representations. Linear probes are trained to predict the stack depth at each token from the model's hidden states, and a principal representation direction is extracted from the probe. Ablation of this direction from the model causes sequential accuracy to drop to 0%, providing strong empirical evidence that the stack representation is not just learned, but is causally necessary for model performance.

1 INTRODUCTION

Mechanistic interpretability aims to understand the blackbox-like behavior of language models (LMs) by identifying features. A feature is a human-interpretable property of the models activation on specific inputs. We use probing classifiers to link these properties to the models activations. Formally, let $f : x \to \hat{y}$ be a language model trained on auto-regressive next-token prediction of some language. A probing classifier $p : f_l(x) \to \hat{z}$ returns a property z from the embeddings of the LM at some layer l. The probing classifier is trained and evaluated on a dataset $D_P = \{f_l(x), z^{(i)}\}$ which is the embedding and the property of sequence at that layer (Belinkov (2022)). While probing classifiers are capable of extracting the latent representations of the language model, *they do not make any statements about causality*. The computational primitives developed during training may or may not be utilized in the computation undertaken by these models.

Formal languages prove to be effective testbeds for performing experiments in interpretability, owing to their mathematically rigorous formulation. All algorithmic tasks can be reduced to a language within a specific class of formal languages, making this line of inquiry worthwhile. Tiwari et al. (2025) empirically show the emergence of stack representations in language models trained on formal languages like Dyck-1 and Shuffle-k. They explicitly do not make any comments on the causality of these stack representations.

This paper builds on previous research by investigating the causal role of stack representations. A probed is train a probe to identify stack depth and then ablate the corresponding component from the model's hidden states during inference. This intervention leads to a near-complete collapse in sequence prediction accuracy, suggesting that the learned stack representation is causally necessary for the model's computation.

2 SETUP

2.1 COUNTER LANGUAGE MODELING

We carry out our experiments over the languages Dyck-1 and Shuffle-k for $k = 2, 4, 6, 8$. Formally, Dyck-1 can be defined as a language over alphabet $\Sigma = \{'(', ')'\}$ with production rules:

$$S \to \begin{cases} \epsilon \\ (S) \\ SS \end{cases}$$

Shuffle is a binary operation over two strings which interleaves two strings in all possible ways. Formally, for strings $u, v \in \Sigma^*$, the shuffle $u \odot v$ is defined inductively as:

$$u \odot \epsilon = \epsilon \odot u = \{u\}$$

$$au \odot bv = \{aw : w \in u \odot bv\} \ \cup \ \{bw : w \in au \odot v\}$$

for $a, b \in \Sigma$. For languages $L_1, L_2 \subseteq \Sigma^*$, define:

$$L_1 \odot L_2 = \bigcup_{u \in L_1, v \in L_2} u \odot v.$$

Let $\{\Sigma_i\}_{i=1}^k$ be pairwise disjoint alphabets, and let $L_i \subseteq \Sigma_i^*$ be Dyck-1 languages over each alphabet. Then, the Shuffle-k language is defined as:

$$\text{Shuffle}_k = L_1 \odot L_2 \odot \cdots \odot L_k.$$

Intuitively, Shuffle_k consists of all strings formed by interleaving k independent Dyck-1 strings while preserving the order of symbols within each string.

2.2 TRANSFORMER ARCHITECTURE

An encoder-only transformer is trained with a causal attention mask for next-token prediction over Shuffle-k strings. The vocabulary consists of k bracket pairs, one padding token and one `BOS` token. Given a prefix, the model predicts the set of valid continuations as a k-hot vector. The model uses embedding dimension $d_{embed} = 32$, hidden dimension $d_{model} = 64$, a single layer with 4 attention heads, feedforward dimension $d_{ffn} = 64$, and learned positional encodings. The decoder is a linear layer with sigmoid activation, and training uses MSE loss with RMSProp (lr 5×10^{-3}, batch size 32, over 10,000 sequences (lengths 2–50, 80/20 training/validation split).

Positional accuracy A_{pos} is defined as the fraction of valid token positions where the model's predicted set is equal to the ground truth set.

$$\text{A}_{\text{pos}} = \frac{1}{|\mathcal{M}|} \sum_{(b,t) \in \mathcal{M}} \mathbf{1}\left[\hat{y}_{b,t} = y_{b,t}\right]$$

Similarly, sequential accuracy of a string is defined as the fraction of strings which are valid in their entirety. One wrong token deems the whole string invalid, and thus this is naturally a more sensitive metric.

$$\text{A}_{\text{seq}} = \frac{1}{B} \sum_{b=1}^{B} \mathbf{1}\left[\forall\, t \in \mathcal{M}_b : \hat{y}_{b,t} = y_{b,t}\right]$$

$$\text{where we define } \hat{y}_{b,t} = \mathbf{1}\left[\sigma(\text{decoder}(h_{b,t})) > 0.5\right]$$

This language model achieves perfect validation accuracy in Shuffle-k for $k = 2, 3, .., 8$ by the 25th epoch, confirming that all representations analyzed in this work belong to a fully converged model.

2.3 PROBE DESIGN

Shuffle-k can be represented as k different stacks, each pertaining to a singular bracket type. An open bracket of type $'i'$ ($open_i$) increases the $depth_i$ (depth of ith stack) by 1, and a closing bracket $close_i$ decreases the stack depth by 1. The validity condition of the string is to have $depth_i = 0 \; \forall i$.

Now, for a string $x = (x_0, x_1, ..., x_{n-1})$ the transformer produces, for a layer $'l'$ at token position $'t'$ a hidden state $h_t^{(l)}$ corresponding to $c_i(t)$, the depth of the ith stack at token position $'t'$. We now have:

$$D_P(l) = \{h_t^{(l)}, c_i(t)\}$$

which is used as the training dataset for the probe. We construct a linear classifier probe:

$$f(h) = Wh + b, W \in \mathbb{R}^{C \times d_{model}}, b \in \mathbb{R}^C$$

where $C =$ number of depth classes. Trained with cross entropy loss:

$$\mathcal{L} = -\sum_{(h,c) \in \mathcal{D}_{\text{train}}} \log \frac{e^{f(h)_c}}{\sum_{j=0}^{C-1} e^{f(h)_j}}$$

using Adam with learning rate 10^{-3}, batch size 32, for 10 epochs, with Xavier uniform weight initialization. A control probe is trained on the dataset $D_{ctrl} = \{h_t^{(l)}, \pi(c_i(t))\}$ [1] and validate on the original validation set. Selectivity is then defined as the difference in the accuracy of the experimental probe and the control probe. High selectivity is an indicator of the stack depth genuinely being encoded in the hidden space vectors.

2.4 INTERVENTION

The linear classifier probe lets us extract the trained probe weight matrix $W \in \mathbb{R}^{C \times d_{model}}$. Performing singular value decomposition

$$W = U \Sigma V^{\dagger}$$

where $U \in \mathbb{R}^{C \times C}$, $\Sigma \in \mathbb{R}^{C \times d_{model}}$, and $V \in \mathbb{R}^{d_{model} \times d_{model}}$

Defining $\mathbf{w} = V_{[0]} \in R^{d_{model}}, ||\mathbf{w}|| = 1$ as the top right singular vector of W, we move forward with ablating the direction of $\mathbf{w}$. For each hidden state $h_t^{(l)}$ at token t in layer l, living in $\mathbb{R}^{d_{model}}$, we perform the following operation:

$$h' = h - \alpha(\mathbf{w} \cdot h)\mathbf{w}, \alpha \in [0, 1]$$

We also ablate random directions $\mathbf{w} \sim Uniform(\mathcal{S}^{d_{model}-1})$ and control directions obtained from the SVDs of classifiers trained on control datasets defined by D_{ctrl}. The ablated states are only passed through the decoder head, the transformer's internal computations are not re-run.

3 RESULTS

The experiment is run on Shuffle-k for $k = 6$. Figure 1 shows ablating $\mathbf{w}$ from each stack and sweeping $\alpha = \{0, 0.25, 0.50, 0.75, 1\}$ causes the positional accuracy to dip, and sequential accuracy to collapse. Furthermore, ablating random directions has no effect on the accuracies of the model. Building upon the discovery of emergent stack representations, these results empirically prove that the stack representations are causally involved in the computation undertaken by the transformer for next-token prediction of counter languages. The sharp threshold structure of the collapse provides evidence of non-redundancy. If depth information were to be distributed in the residual stream, partial ablation would be compensated for by alternative encodings, ensuring a gradual curve.

[1] where $\pi(x)$ denotes a random perturbation of the training labels.

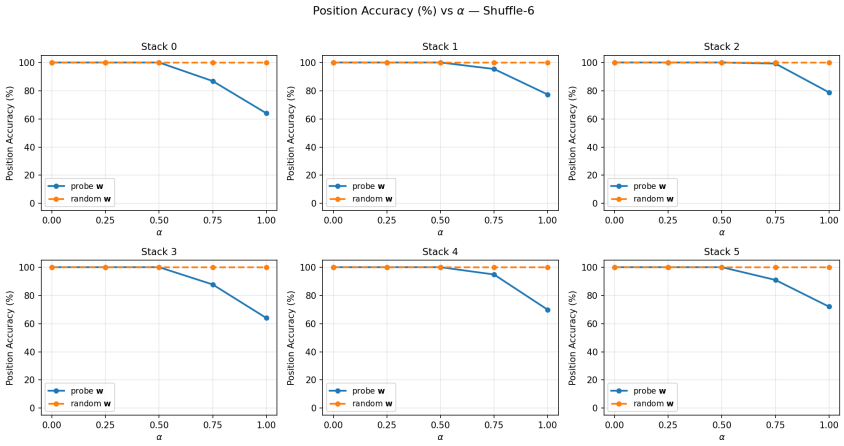

Figure 1: Position accuracy as a function of ablation strength α.

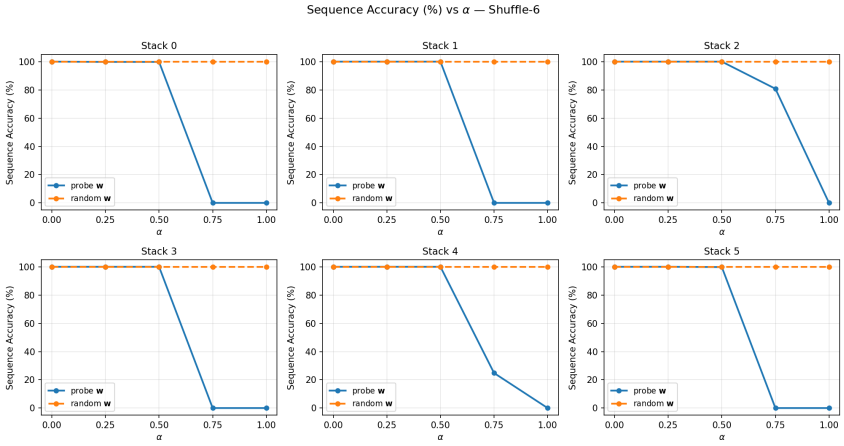

Figure 2: Sequence accuracy as a function of ablation strength α.

4 Future Works

While these results provide evidence of stack representations being causally important for model performance, the broader task of mechanistic interpretability remains open - *algorithms.* How the transformer tackles the manipulation of emergent computational primitives like stacks and counters while modeling formal languages might be an important avenue demanding exploration. Furthermore, the role of different architectures in the emergence of representations, their redundancy, causality, and their relationships remains an interesting direction.

References

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, March 2022. doi: 10.1162/coli_a_00422. URL https://aclanthology.org/2022.cl-1.7/.

Utkarsh Tiwari, Aviral Gupta, and Michael Hahn. Emergent stack representations in modeling counter languages using transformers. In *ICLR 2025 Workshop on World Models: Understanding, Modelling and Scaling*, 2025. URL https://openreview.net/forum?id=o5xTyDilNs.